



كلية هندسة الذكاء الاصطناعي
FACULTY OF ARTIFICIAL INTELLIGENCE ENGINEERING



Syrian Private University

Faculty of Engineering

Department of Artificial Intelligence Engineering and Data Science

Graduation Project 1

A Personal Assistant for Arabic Text Summarization Using Artificial Intelligence Techniques

مساعد شخصي لتلخيص النصوص العربية باستخدام تقنيات الذكاء الاصطناعي

This thesis was prepared to complete the Graduation Project 1 course in Artificial Intelligence Engineering and Data Science.

Prepared by

Mosaab Abdul Latif Yousef - Mohammed Aqeed Amin - Muhand Adel Al-Salehani

مصعب عبد اللطيف يوسف - محمد عكيد أمين - مهند عادل الصالحاني

Supervised by

Dr. Kadan Jomaa - Eng. Anas Abdul Aziz

د. كادان جمعة - م. أنس عبد العزيز

Damascus - Syrian Private University

2026/5/2

SUPERVISOR CERTIFICATION

I certify that the preparation of this project, entitled “A Personal Assistant for Arabic Text Summarization Using Artificial Intelligence Techniques,” prepared by Mosaab Abdul Latif Yousef, Mohammed Aqeed Amin, and Muhand Adel Al-Salehani, was carried out under my supervision at the Faculty of Engineering, Department of Artificial Intelligence Engineering and Data Science, in partial fulfillment of the requirements of Graduation Project 1.

Supervisor Name:.....

Date:.... /.... / 2026

Signature:.....

Declaration

- We declare that the work and results presented in this thesis are the outcome of our personal effort and were completed under the guidance of our supervisors. Any information, result, paragraph, figure, diagram, or external material taken from other sources has been clearly attributed to its original source in the text and in the references section.
- We also declare that the data and information used in this thesis were obtained through legitimate and proper methods, and that the work presented here has not been used previously, nor is it currently being used, to obtain any other academic degree.

Signatures:

Mohammed Aqeed Amin: _____

Mosaab Abdul Latif Yousef: _____

Muhand Adel Al-Salehani: _____

Date: 1 / 5 / 2026

Acknowledgment

We would like to express our sincere gratitude to our supervisors, Dr. Kadan Jomaa and Eng. Anas Abdul Aziz, for their continuous guidance, valuable feedback, and academic support throughout the stages of this project. Their observations helped improve the scientific quality, organization, and practical direction of this work.

We also extend our appreciation to the teaching staff of the Faculty of Engineering at the Syrian Private University, and to everyone who provided academic or moral support during the preparation of this thesis.

Arabic Translation

نتقدم بجزيل الشكر والتقدير إلى مشرفينا د. كادان جمعة وم. أنس عبد العزيز على توجيههم المستمر وملاحظاتهم العلمية القيمة ودعمهم الأكاديمي خلال مراحل هذا المشروع. كما نشكر الكادر التدريسي في كلية الهندسة في الجامعة السورية الخاصة، وكل من قدم لنا الدعم العلمي أو المعنوي أثناء إعداد هذه الأطروحة.

Dedication

We dedicate this work to our families, who have always been our first source of support, to our teachers who guided us with knowledge, and to our colleagues and everyone who supported us throughout this academic journey.

Arabic Translation

نهدي هذا العمل إلى أهلنا الذين كانوا الداعم الأول في مسيرتنا العلمية، وإلى أساتذتنا الذين منحونا المعرفة والتوجيه، وإلى زملائنا وكل من ساندنا خلال هذه الرحلة الأكاديمية.

Arabic Abstract

- يهدف هذا المشروع إلى تصميم وتطوير مساعد ذكي لتلخيص النصوص العربية الطويلة باستخدام نموذج UBC-NLP/AraT5v2-base-1024 المعتمد على بنية المحولات من نوع Encoder-Decoder. تتبع أهمية المشروع من الحاجة إلى تقليل الوقت والجهد اللازمين لقراءة النصوص العربية الطويلة، وخاصة في المجالات التعليمية والبحثية، مع الحفاظ على المعنى الأساسي وتقليل التكرار. وتظهر الدراسات الحديثة أن التلخيص العربي ما زال يواجه تحديات مرتبطة بغنى اللغة العربية صرفياً ودلالياً، وبقلة الموارد المعيارية الخاصة بالتقييم والتدريب [١]، [٢].
- اعتمدت المنهجية على مجموعة بيانات (ASDs) Arabic Summarization Dataset، وهي مجموعة بيانات توفر تقسيماً جاهزاً إلى بيانات تدريب وتحقق واختبار بأحجام ٢٠٧٥ و ٢٥٩ و ٢٦٠ على التوالي. شملت المعالجة المسبقة تنظيف الفراغات الزائدة وإزالة المحارف غير المرئية الخاصة باتجاه النص، ثم تقطيع النصوص باستخدام المحلل اللغوي الخاص بالنموذج. تم تدريب النموذج لمدة ١٥ حقبة باستخدام معدل تعلم $3e-5$ ، وحجم دفعة ٢، وخطوات تراكم تدرج ٤، وقيمة Weight Decay تساوي ٠.٠١.
- عولجت النصوص الطويلة باستخدام تقنية Chunking بحجم ٨٥٠ رمزاً وتداخل ٨٠ رمزاً، مع عدد أقصى من جولات التلخيص الهرمي يساوي ٣. كما تم توليد الملخصات باستخدام Beam Search بعدد حزم ٤، وطول أدنى ١٥، وطول أقصى ١٢٨، وقيمة length penalty تساوي ١.٠، ومنع تكرار n-grams بحجم ٣. أظهرت نتائج الاختبار النهائي القيم التالية: ROUGE-1 = 7.4982، ROUGE-L = 7.5458، BERTScore = 77.7619، METEOR = ٠.٣٨٤٦، chrF = 39.6919، و Semantic Similarity = 83.5240. كما تم بناء واجهة Gradio لتلخيص النصوص العربية وملفات PDF النصية فقط، مع عدم دعم ملفات PDF المصورة لعدم وجود OCR في الكود.
- الكلمات المفتاحية: التلخيص العربي، معالجة اللغات الطبيعية، المحولات، AraT5v2، التلخيص التجريدي، Gradio، Chunking.

Abstract

- This thesis aims to design and develop an intelligent assistant for summarizing long Arabic texts using the UBC-NLP/AraT5v2-base-1024 model, which is based on the Transformer encoder-decoder architecture. The project addresses the increasing need to reduce the time and effort required to read long Arabic texts, especially in educational and research contexts, while preserving the core meaning and reducing redundancy. Recent Arabic summarization studies emphasize that Arabic NLP still faces linguistic complexity, limited standardized resources, and evaluation challenges [1], [2].
- The methodology relies on the Arabic Summarization Dataset (ASDs), which provides official train, validation, and test splits of 2075, 259, and 260 samples, respectively. Preprocessing includes whitespace normalization, removal of hidden bidirectional characters, and tokenization using the model-specific tokenizer. The model was fine-tuned for 15 epochs with a learning rate of $3e-5$, a batch size of 2, gradient accumulation steps of 4, and a weight decay of 0.01.
- Long texts were handled through chunking with a chunk token size of 850 and an overlap of 80 tokens, with up to 3 hierarchical summarization rounds. Summaries were generated using beam search with 4 beams, a minimum summary length of 15, a maximum summary length of 128, a length penalty of 1.0, and a no-repeat n-gram size of 3. Final test results were as follows: ROUGE-1 = 7.4982, ROUGE-2 = 0.3846, ROUGE-L = 7.5458, BERTScore = 77.7619, METEOR = 26.8728, chrF = 39.6919, and Semantic Similarity = 83.5240. A Gradio interface was also developed to summarize Arabic text and text-based PDF files only; scanned PDFs are not supported because OCR was not implemented.
- Keywords: Arabic text summarization, natural language processing, Transformers, AraT5v2, abstractive summarization, chunking, Gradio.

Table of Contents

SUPERVISOR CERTIFICATION	2
Declaration	3
Acknowledgment	4
Dedication	5
Arabic Abstract	6
Abstract	6
List of Tables	8
List of Figures	9
List of Equations	9
List of Terms and Symbols	9
Chapter 1: Introduction - تمهيد	12
Chapter 2: Theoretical Background - الدراسة النظرية	15
Chapter 3: Literature Review - الدراسة المرجعية	18
Chapter 4: Research Methodology - منهج البحث	24
Chapter 5: Results and Evaluation - نتائج البحث	34
Chapter 6: Conclusion - الخاتمة	40
Appendix A: Notebook Implementation Summary	41
Appendix B: Reproducibility and Deployment Notes	44
Appendix C: Consistency Notes Between Thesis and Implementation	45
References	46

List of Tables

Table 0.1: Key technical terms and symbols used in the thesis.	10
Table 3.1: Summary of selected previous studies in Arabic text summarization.	18
Table 3.2: Detailed analysis of selected references and their relation to this project.	21
Table 3.3: Comparative analysis of major summarization approaches.	22
Table 4.1: Official splits of the Arabic Summarization Dataset (ASDs).	25
Table 4.2: Actual training configuration used in the notebook implementation.	26

Table 4.3: Generation parameters used for summary decoding.	27
Table 4.4: Long-text chunking configuration.	28
Table 4.5: Detailed code workflow and its role in the implemented system.	32
Table 5.1: Evaluation metrics and equations used in the project.	35
Table 5.2: Final test results obtained on the official ASDs test split.	36
Table 5.3: Qualitative demonstration of long-text summarization.	38
Table 5.4: Analytical comparison between the proposed system and selected previous work.	39

List of Figures

Figure 2.1: AraT5v2 encoder-decoder architecture used in the proposed system.	16
Figure 4.1: End-to-end pipeline of the Arabic text summarization system.	27
Figure 4.2: Representative Gradio interface for Arabic text and text-based PDF summarization.	29
Figure 5.1: Bar chart of the final evaluation metrics.	36
Figure 5.2: Training and validation loss curve during fine-tuning.	37

List of Equations

Equation (5.1): ROUGE-N recall-based overlap.	34
Equation (5.2): ROUGE-L longest common subsequence recall.	34
Equation (5.3): BERTScore F1 formulation.	34
Equation (5.4): Semantic similarity using cosine similarity .	34

List of Terms and Symbols

Table 0.1: Key technical terms and symbols used in the thesis.

English Term	Arabic Translation	Abbreviation	Meaning / Context
Natural Language Processing	معالجة اللغات الطبيعية	NLP	Field that enables computers to understand and generate human language.
Automatic Text Summarization	التلخيص التلقائي للنصوص	ATS	Producing a shorter version of a source text while preserving important information.
Abstractive Summarization	التلخيص التجريدي	-	Generating a new summary rather than extracting sentences directly.
Extractive Summarization	التلخيص الاستخراجي	-	Selecting important sentences or phrases from the source text.
Transformers	المحولات	-	Deep learning architecture based on attention mechanisms.
Encoder-Decoder	المشفّر وفكّ التشفير	-	The encoder reads the input text and the decoder generates the output summary.
Self-Attention	الانتباه الذاتي	-	A mechanism that relates each token to other tokens in the same sequence.
Cross-Attention	الانتباه المتقاطع	-	Allows the decoder to focus on relevant encoder representations while generating.
AraT5v2	نموذج AraT5v2	-	Arabic pretrained sequence-to-sequence model used in the implementation.
AraBART	نموذج AraBART	-	Important related Arabic model

Summarizing texts in Arabic

			discussed only in the literature review.
Fine-Tuning	الضبط الدقيق	-	Adapting a pretrained model to a specific downstream task.
Tokenizer	المحلل اللغوي	-	Converts text into token IDs that the model can process.
Chunking	تقسيم النص إلى أجزاء	-	Processing long texts by splitting them into overlapping token segments.
Beam Search	البحث الشعاعي	-	A decoding strategy that explores multiple candidate sequences.
ROUGE	مقياس روج	ROUGE	Measures lexical overlap between generated and reference summaries.
BERTScore	مقياس بيرت سكور	-	Measures semantic similarity using contextual embeddings.
METEOR	مقياس ميتيور	METEOR	Evaluates flexible word matching and similarity beyond exact overlap.
chrF	مقياس chrF	-	Character-level F-score useful for morphologically rich languages.
Semantic Similarity	التشابه الدلالي	-	Measures meaning-level closeness between two summaries.
Text-Based PDF	نصي PDF ملف	-	A PDF from which text can be extracted directly; scanned PDF requires OCR.

Chapter 1: Introduction — الفصل الأول: تمهيد

This chapter introduces the general context of Arabic text summarization, the scientific problem, the research objectives, the contribution of the project, the beneficiaries, and the organization of the thesis. It also clarifies the practical scope of the implemented system so that the thesis remains aligned with the notebook implementation and with the graduation-project template.

1.1 Introduction

The rapid growth of Arabic digital content has created a major challenge for students, researchers, and readers who need to extract essential information quickly and accurately. Educational, academic, and news documents are becoming longer and more numerous, which increases the need for intelligent tools capable of producing concise and meaningful summaries. Long-text summarization is especially important because long Arabic documents may contain several sections, repeated ideas, and domain-specific vocabulary that cannot be handled efficiently by manual reading alone [1].

Traditional human summarization requires considerable time and effort and may be affected by subjectivity, inconsistency, and the reader's background knowledge. This makes manual summarization impractical when dealing with large amounts of Arabic documents or long educational materials. Automatic Text Summarization provides a practical solution by producing a shorter version of the input while preserving the main ideas. In Arabic, however, this task is harder than in many other languages because the same root can generate many surface forms, and because syntactic order and contextual meaning may vary across sentences [1], [2].

With the development of deep learning and Transformer-based models, abstractive summarization has become more capable of understanding context and generating coherent summaries rather than simply extracting sentences from the source. This project focuses on building a personal assistant for Arabic text summarization using UBC-NLP/AraT5v2-base-1024. The system supports Arabic text input and text-based PDF files, applies preprocessing and long-text chunking, generates summaries using beam search, and provides a Gradio interface for practical use. The system therefore combines a research-oriented model with an end-user application layer.

1.2 Scientific Problem

Educational and research institutions in the Arab region face a growing challenge in managing and understanding large volumes of Arabic digital content. The lack of practical and reliable Arabic summarization tools increases the time needed to read and analyze long documents. Existing research has achieved important progress, but many studies focus mainly on reporting model results without providing a complete assistant that can be used directly by students or researchers.

Arabic also presents linguistic challenges due to its rich morphology, derivational patterns, prefixes, suffixes, and contextual ambiguity. These characteristics make it difficult for a summarization system to preserve the intended meaning while producing a concise output. Previous Arabic summarization work has repeatedly identified limited datasets, scarce annotated corpora, and the need for Arabic-aware processing as important limitations [1], [2].

Another technical challenge is the limited input length of Transformer models. Long documents may exceed the model's maximum token length, which requires an additional strategy such as chunking. Generative models may also produce unsupported information if they are not properly trained and evaluated. Therefore, the main research problem is the need for a reliable Arabic summarization system that can summarize long Arabic texts while preserving meaning, reducing redundancy, handling long inputs, and supporting practical use through a simple interface.

1.3 Research Objectives

The main objective of this project is to build an intelligent assistant for summarizing long Arabic texts and text-based PDF files using AraT5v2-base-1024 and modern NLP techniques. The objectives below are written to match the notebook implementation exactly and to avoid unsupported claims.

- Develop an abstractive Arabic summarization model using UBC-NLP/AraT5v2-base-1024.
- Apply the actual preprocessing steps implemented in the notebook: whitespace normalization, removal of hidden directional characters, and tokenization.
- Use the official ASDs dataset split: 2075 training samples, 259 validation samples, and 260 test samples.
- Fine-tune the model for 15 epochs using a learning rate of $3e-5$ and the training configuration implemented in the code.

- Handle long texts using chunking with 850 tokens per chunk and 80 tokens of overlap.
- Evaluate the model using ROUGE-1, ROUGE-2, ROUGE-L, BERTScore, METEOR, chrF, and Semantic Similarity.
- Build a Gradio interface for Arabic text and text-based PDF summarization.

1.4 Research Contribution

The contribution of this project is the development of an applied Arabic summarization assistant that combines a pretrained Arabic sequence-to-sequence model, task-specific fine-tuning, long-text chunking, beam-search generation, multi-metric evaluation, and a user-facing interface. The contribution is not presented as a new neural architecture; rather, it is an applied and reproducible system that integrates available Arabic NLP resources into a usable assistant.

The project also clarifies the practical limitations of Arabic abstractive summarization, including input length constraints, limited Arabic summarization datasets, possible hallucination, and the need for semantic evaluation beyond surface-level lexical overlap. This is important because ROUGE alone may not capture semantic similarity when an abstractive model generates valid wording that differs from the reference summary [6], [6], [6].

1.5 Beneficiaries

- Students who need to summarize lectures, textbooks, and long educational materials.
- Researchers who need quick access to the main ideas of papers, reports, and academic documents.
- Teachers and educational institutions that manage Arabic learning content.
- Content editors and journalists who work with long Arabic texts.
- Developers and researchers interested in Arabic NLP applications.

1.6 Thesis Organization

This thesis is organized into six chapters. Chapter 1 introduces the problem, objectives, contribution, and beneficiaries. Chapter 2 presents the theoretical background of Arabic NLP, summarization, Transformers, and AraT5v2. Chapter 3 reviews related studies and identifies the research gap. Chapter 4 explains the research methodology, dataset, preprocessing, model training, long-text handling, and interface. Chapter 5 presents the evaluation results and analysis. Chapter 6 concludes the work and discusses challenges and future directions. The appendices summarize the notebook implementation, reproducibility notes, and consistency checks between the thesis and the code.

Chapter 2: Theoretical Background — الفصل الثاني: الدراسة النظرية

2.1 Natural Language Processing

Natural Language Processing (NLP) is a field that combines artificial intelligence, linguistics, and computer science to enable computers to analyze, understand, and generate human language. In this project, the focus is Arabic, a morphologically rich language that introduces additional challenges for text processing and summarization. These challenges include word derivation, attachment of prefixes and suffixes, optional diacritics, and ambiguity that often requires understanding the surrounding context [1], [2].

2.2 Automatic Text Summarization

Automatic Text Summarization aims to produce a shorter version of a source text while preserving its most important information. It is commonly divided into extractive and abstractive summarization. Extractive summarization selects existing sentences from the source, while abstractive summarization generates new wording that may not appear directly in the original text. Hybrid summarization combines both ideas, often by extracting important content first and then rewriting or compressing it.

This project adopts the abstractive approach because it is closer to human summarization and can produce more coherent summaries than simple sentence extraction. However, abstractive summarization is more difficult because it requires the model to understand context, preserve meaning, and avoid generating unsupported information. This is why the thesis uses multiple evaluation metrics rather than relying only on a single lexical metric.

2.3 Transformer Architecture

Transformer models rely on attention mechanisms that allow the model to represent relationships between tokens regardless of their distance in the sequence. The original Transformer architecture introduced the attention-based encoder-decoder approach that became the foundation of many modern NLP models [6]. Unlike recurrent models, Transformers can process tokens in parallel and can capture long-range dependencies more effectively.

The encoder-decoder structure is particularly suitable for summarization. The encoder reads and represents the input text, while the decoder generates the summary token by token. Self-attention captures relationships inside the input sequence, and cross-attention allows the decoder to focus on relevant parts of the source text during generation. This structure is the reason sequence-to-sequence models such as AraBART and AraT5 are widely used for abstractive summarization [3], [6].

2.4 AraT5v2-base-1024

The practical model used in this project is UBC-NLP/AraT5v2-base-1024. It is an Arabic pretrained sequence-to-sequence model suitable for generation tasks such as summarization. The model can handle inputs up to 1024 tokens, which is useful for medium-length texts, but long documents still require chunking. Fine-tuning allows the model to adapt its general Arabic language knowledge to the specific task of summarization by learning from pairs of articles and reference summaries [6], [6].

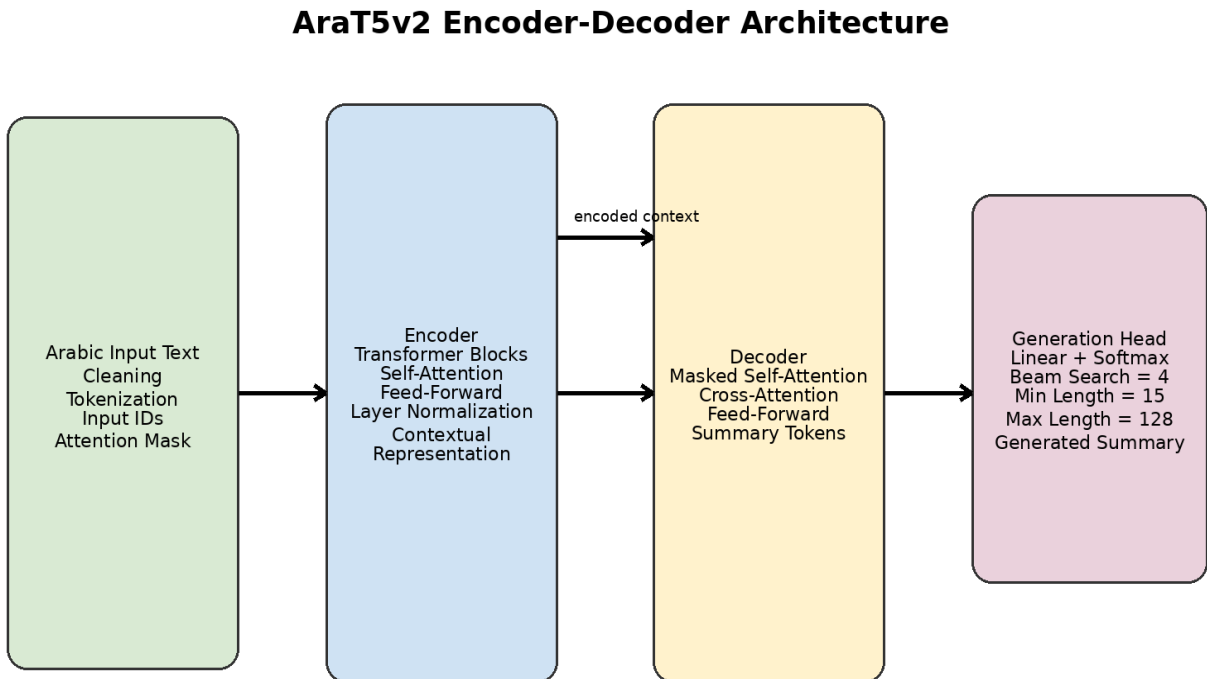


Figure 2.1: AraT5v2 encoder-decoder architecture used in the proposed system.

2.5 AraBART in the Literature Review

AraBART is discussed as an important Arabic abstractive summarization model in the literature review. Kamal Eddine et al. introduced AraBART as an Arabic pretrained sequence-to-sequence model in which the encoder, decoder, and connection weights are pretrained end-to-end using a BART-style denoising objective [3]. However, AraBART should not be described as the model used in the practical implementation unless a separate AraBART experiment is actually performed. In this thesis, AraBART remains a reference model for comparison, while AraT5v2 is the implemented model.

2.6 Arabic Text Preprocessing

The preprocessing used in the notebook is intentionally limited and should be described accurately. The implemented cleaning function normalizes repeated whitespace, removes hidden left-to-right and right-to-left direction marks, and trims the text. It does not implement advanced normalization such as full diacritic removal, hamza normalization, link removal, stemming, or lemmatization unless these operations are added later to the code.

After cleaning, the texts and summaries are tokenized using the AraT5 tokenizer. The maximum input length is 1024 tokens and the maximum target length is 128 tokens. Dynamic padding is handled during batching by DataCollatorForSeq2Seq. This description is important because the thesis must reflect the actual notebook rather than a general preprocessing pipeline from the literature.

2.7 Long-Text Handling Using Chunking

Because the model has an input limit, long texts are processed using chunking. The implementation uses a chunk token size of 850 and an overlap of 80 tokens. Each chunk is summarized separately, then the partial summaries are merged and summarized again when needed. The maximum number of hierarchical summarization rounds is 3. This design is aligned with long-document summarization discussions in the literature, where segmentation is commonly used to make long input manageable [1].

2.8 Loss Function and Optimization

Sequence-to-sequence summarization models are trained using Cross-Entropy Loss, which measures the difference between the predicted token distribution and the correct reference token. The loss value is not a percentage; it is a mathematical training objective used to update the model weights. A lower loss generally indicates that the model is learning to predict the reference summary tokens more accurately, but it does not directly represent an accuracy percentage.

The optimizer used through the Hugging Face Trainer is AdamW. In the notebook, AdamW is used indirectly through Seq2SeqTrainer to update the Transformer weights during fine-tuning. Weight decay was set to 0.01 to reduce overfitting, gradient accumulation was used to simulate a larger effective batch size without exceeding GPU memory, and early stopping was added to stop training when validation performance no longer improves. These settings reduce memory pressure and improve reproducibility in Google Colab.

Chapter 3: Literature Review — الفصل الثالث: الدراسة المرجعية

This chapter reviews recent and relevant studies in Arabic text summarization and general automatic summarization. The purpose is not only to list related papers, but also to explain what each study contributed, what methods and metrics it used, what results it reported, and how its limitations motivate the proposed system. The final reference list was limited to the project sources and code-related sources used in this thesis, so unsupported additional references were removed.

3.1 Previous Studies in Related Fields

Table 3.1: Summary of selected previous studies in Arabic text summarization.

Study	Year	Model / Method	Metrics	Main Finding
Magdy et al.	2024	Survey of Arabic long-text summarization	Comparative analysis	Highlighted the importance of long-document segmentation, hybrid methods, datasets, and evaluation gaps [1].
Abdul Salam et al.	2024	MSG-ATS semantic graph + deep learning	ROUGE and manual evaluation	Improved over Word2Vec baselines by modeling syntactic, semantic, and contextual relations [2].
Kamal Eddine et al.	2022	AraBART pretrained Arabic seq2seq model	ROUGE, BERTScore, human evaluation	Introduced AraBART and reported strong performance on Arabic abstractive summarization datasets [3].
Masri et al.	2024	AraBART, MT5, AraT5, mBART50 for textbooks	ROUGE and expert rating	AraBART achieved the highest expert rating, while mBART50 had the highest ROUGE-2 in that study [4].
Merniz et al.	2021	Thematic annotation + graph representation + modified PageRank	ROUGE-1/2/L/SU4	Reduced redundancy in multi-document summarization through topic-based graphs [5].
Zhang et al.	2025	Comprehensive survey including LLM-based ATS	Comparative taxonomy	Classified modern LLM-based summarization into prompting, RAG, fine-tuning...

3.1.1 AraBART: Arabic Pretrained Sequence-to-Sequence Summarization

Kamal Eddine et al. proposed AraBART, a pretrained Arabic sequence-to-sequence model designed for abstractive summarization [3]. The main contribution of this work is that the encoder, decoder, and cross-attention connections are pretrained end-to-end, unlike BERT-based encoder-decoder settings where only the encoder may be pretrained. This is important because summarization is a generative task and therefore benefits from a pretrained decoder as well as a pretrained encoder.

The study evaluated AraBART against strong baselines such as mBART25, mT5base, AraT5base, and CAMELBERT2CAMELBERT. It reported ROUGE-1, ROUGE-2, ROUGE-L, and BERTScore. In the macro-average results across the tested datasets, AraBART achieved ROUGE-1 = 42.4, ROUGE-2 = 28.8, ROUGE-L = 40.3, and BERTScore = 69.8 [3]. The strength of this paper is its direct comparison with multiple pretrained sequence-to-sequence baselines and its use of both automatic and human evaluation. Its limitation for the present thesis is that it focuses on model benchmarking rather than building a practical assistant interface or supporting text-based PDF input.

3.1.2 MSG-ATS: Multi-Level Semantic Graph for Arabic Text Summarization

Abdul Salam et al. introduced MSG-ATS, a multi-level semantic graph framework for Arabic abstractive summarization [2]. The model constructs semantic graphs from Arabic text, embeds graph representations, and uses deep learning to generate summaries. This direction is valuable because it explicitly models syntactic, semantic, and contextual relationships that may be lost in simple word embedding methods.

The paper reports that MSG-ATS outperformed the Word2Vec baseline by 42.4% in precision, 23.8% in recall, and 38.3% overall [2]. The strength of this approach is its attention to Arabic semantic structure and graph relationships. However, the method is more complex than fine-tuning a pretrained Transformer model and requires additional graph construction and feature engineering. In this thesis, MSG-ATS is used as evidence that Arabic summarization benefits from semantic-aware modeling, while the implemented system chooses a more practical Transformer fine-tuning pipeline.

3.1.3 Transformer Models for Arabic Science Textbooks

Masri et al. studied Arabic educational summarization using biology textbooks for the 11th and 12th grades in the Palestinian curriculum [4]. The study is close to the motivation of this thesis because it focuses on educational Arabic content rather than only news articles. The dataset contained 111 rows with fields such as unit title, lesson title, section content, questions, and expert summary. The authors trained and evaluated AraBART, MT5, AraT5, and mBART50 using ROUGE and expert evaluation.

The reported ROUGE results showed AraBART with ROUGE-1 = 0.2462, ROUGE-2 = 0.1184, and ROUGE-L = 0.2462; mBART50 with ROUGE-1 = 0.2431, ROUGE-2 = 0.1392, and ROUGE-L = 0.2431; AraT5 with ROUGE-1 = 0.2065, ROUGE-2 = 0.1153, and ROUGE-L = 0.2065; and MT5 with ROUGE-1 = 0.0566 and ROUGE-2 = 0 [4]. The expert ratings were AraBART = 8.409, mBART50 = 8.18, AraT5 = 7.727, and MT5 = 7.17 [4]. Therefore, it is inaccurate to claim that AraT5 was the best model in this study; AraBART had the highest expert rating, while mBART50 achieved the highest ROUGE-2. This correction was added to the thesis to avoid a reference error.

3.1.4 Thematic Annotation and PageRank for Multi-Document Arabic Summarization

Merniz et al. proposed a multi-document Arabic summarization method based on thematic annotation, graph representation, and a modified PageRank algorithm [5]. The correct year for this paper is 2021, not 2024. The method segments documents by topic using thematic annotation, represents topic segments as graphs, and applies modified PageRank using cosine similarity as edge weights. This design aims to reduce redundancy, which is a major problem in multi-document summarization.

The study evaluated its method using ROUGE-1, ROUGE-2, ROUGE-L, and ROUGE-SU4 on topics such as economics, health, and politics. For example, the health topic reached ROUGE-1 recall = 0.59024 and ROUGE-2 recall = 0.36019, while the economics topic reached ROUGE-SU4 recall = 0.39250 [5]. The strength of this work is its graph-based reduction of redundancy across multiple documents. Its limitation for this thesis is that it is extractive and multi-document oriented, while the proposed system is an abstractive single-input assistant with long-text chunking.

3.1.5 Survey on Arabic Long-Text Summarization

Magdy et al. presented a survey on Arabic long-text summarization strategies [1]. The study emphasizes that long-document summarization requires dividing lengthy texts into smaller segments and that hybrid methods can be useful because they combine the precision of extractive summarization with the flexibility of abstractive summarization. It also discusses Arabic challenges such as morphology, syntax, semantics, lexical richness, and the limited availability of datasets for Arabic long documents.

This survey supports the design decision in the current thesis to use chunking for long inputs. It also supports the future-work recommendation to collect more educational and scientific Arabic data. The limitation of a survey paper is that it does not provide a single implementation to reproduce, but it is valuable for identifying gaps and research directions.

3.1.6 LLM-Based Automatic Text Summarization

Zhang et al. reviewed automatic text summarization from conventional methods to LLM-based methods [6]. The paper classifies LLM-based summarization into prompt engineering, retrieval-augmented generation, fine-tuning, and knowledge distillation. It also highlights a key difference

between conventional systems and LLM-based systems: LLMs can often work in a more end-to-end manner, but they may still suffer from hallucination and factual inconsistency.

This reference is important for explaining why the current thesis does not claim that LLMs are always superior. LLMs offer strong fluency and flexible instructions, but they may require higher computational cost, raise privacy concerns, and produce unsupported content. Therefore, the implemented system uses a fine-tuned Arabic sequence-to-sequence model and keeps LLM-based prompting as a future extension rather than as the current implementation.

3.2 Detailed Analysis of References

Table 3.2: Detailed analysis of selected references and their relation to this project.

Reference	Strengths	Limitations	How this thesis uses it
AraBART [3]	Strong Arabic seq2seq baseline; end-to-end pretrained encoder and decoder; evaluated with automatic and human metrics.	Does not provide the practical Gradio/PDF assistant implemented in this thesis.	Used as a reference model and comparison point, not as the implemented model.
MSG-ATS [2]	Models Arabic syntax, semantics, and context using semantic graphs and GNN/deep learning components.	More complex pipeline; requires graph construction and semantic feature engineering.	Supports the argument that Arabic summarization needs semantic-aware evaluation and modeling.
Masri et al. [4]	Closest educational motivation; compares AraBART, mBART50, AraT5, and MT5 on Arabic biology textbook content.	Small dataset of 111 rows; model ranking depends on metric; AraBART is not AraT5.	Used to justify educational use cases and correct model-comparison statements.
Merniz et al. [5]	Useful thematic graph approach for reducing redundancy in multi-document Arabic summarization.	Extractive and multi-document oriented; not a neural abstractive assistant.	Used to motivate redundancy handling and analytical comparison.
Magdy et al. [1]	Summarizes long-text Arabic gaps, applications, datasets, preprocessing, and segmentation.	Survey only; does not implement one proposed system.	Used to justify chunking and future dataset expansion.
Zhang et al. [6]	Explains modern LLM-based summarization categories and evaluation	General ATS survey; not Arabic-specific in all sections.	Used to discuss future work and hallucination risks.

3.3 Modern Techniques and Frameworks

Modern Arabic summarization research increasingly relies on pretrained Transformer models, including AraT5, AraBART, mBART, and mT5. These models are suitable for

abstractive summarization because they generate text rather than only selecting sentences. Other directions include graph-based summarization, thematic segmentation, prompt engineering with large language models, and hybrid extractive-abstractive pipelines [1], [2], [6].

From an implementation perspective, the notebook uses Hugging Face Transformers, Datasets, Evaluate, PyTorch, pypdf, SentenceTransformers, and Gradio to build a reproducible workflow. These tools are described as implementation tools rather than as separate experimental baselines. The implemented system is therefore different from purely theoretical or survey-based studies because it includes training, inference, evaluation, long-text handling, PDF text extraction, and a user-facing interface.

3.4 Comparative Analysis

Table 3.3: Comparative analysis of major summarization approaches.

Approach	Techniques	Strengths	Weaknesses
Traditional Extractive Methods	TF-IDF, TextRank, PageRank	Fast, simple, interpretable, and require fewer computational resources.	Do not generate new phrasing and may produce disconnected summaries.
Arabic Abstractive Models	AraT5, AraBART, Encoder-Decoder	Good fluency and contextual understanding; suitable for generation tasks.	Require data, computational resources, and careful evaluation.
Graph-Based Methods	Semantic graphs, thematic graphs, GNNs, PageRank	Can represent relations and reduce redundancy.	More complex to build and tune; often needs feature engineering.
Hybrid Methods	Extractive + Abstractive, thematic segmentation	Useful for long documents and coverage.	More complex to design and evaluate.
Large Language Models	LLMs, prompt engineering, RAG	High fluency and flexible instructions.	Costly, may hallucinate, and may raise privacy concerns.
Proposed System	AraT5v2 + Chunking + Gradio	Practical Arabic assistant with long-text and text-based PDF support.	Still needs stronger ROUGE results and broader human evaluation.

3.5 Research Gap and Proposed Innovation

- The literature shows that Arabic summarization still faces challenges related to long documents, limited specialized datasets, hallucination, and insufficient practical interfaces for end users. Many studies focus on model performance but do not always provide a complete usable tool for text and document summarization. In addition, some systems focus on extractive methods and therefore cannot generate new coherent wording, while purely generative systems may still need evaluation for factual faithfulness.
- The proposed project addresses these gaps by combining AraT5v2 fine-tuning, official ASDs dataset splits, long-text chunking, beam-search generation, multi-metric evaluation, and a Gradio interface for text and text-based PDF summarization. The innovation is therefore applied and integrative: it does not claim a new Transformer architecture, but it turns Arabic abstractive summarization into a reproducible assistant that can be demonstrated and extended.

Chapter 4: Research Methodology — الفصل الرابع: منهج البحث

4.1 Introduction

- This chapter explains the practical methodology used to build the Arabic summarization assistant. It covers the dataset, preprocessing, model architecture, training configuration, generation settings, long-text summarization, and the user interface. Every value in this chapter was checked against the notebook implementation to avoid inconsistencies such as reporting the wrong number of epochs or the wrong learning rate.

4.2 Traditional Methods for the Problem

- Earlier summarization systems often relied on extractive approaches such as TF-IDF, TextRank, and PageRank. These methods identify important words or sentences and select the most relevant ones. Although they are computationally efficient and interpretable, they cannot rewrite content or synthesize information from different parts of the text. For Arabic, extractive methods may also produce summaries that are grammatically disconnected when selected sentences come from distant parts of a document.
- The proposed system uses a Transformer-based abstractive model instead because it can generate a new summary that is more coherent and closer to human writing. However, because abstractive generation is harder to control, the system also uses beam search, no-repeat n-gram control, and several evaluation metrics to analyze both lexical overlap and semantic similarity.

4.3 Dataset

- The dataset used in the implementation is Arabic Summarization Dataset (ASDs), loaded from Hugging Face using the name karimraouf/Arabic-Summarization-Dataset-AsDs [6]. The dataset contains four fields: title, article, summary, and topic. The implementation uses the article field as input and the summary field as target output. The title and topic fields can be useful for analysis, but they are not the target of the implemented summarization training pipeline.
- The dataset already provides official splits; therefore, the final thesis must not state that an 80/20 split was used. The correct split is 2075 samples for training, 259 samples for validation, and 260 samples for testing. Using the official split improves reproducibility because the same samples are used each time the notebook is executed.

Table 4.1: Official splits of the Arabic Summarization Dataset (ASDs).

Split	Number of Samples	Fields
Train	2075	title, article, summary, topic
Validation	259	title, article, summary, topic
Test	260	title, article, summary, topic

4.4 Preprocessing

- The implemented preprocessing includes converting input values to strings, normalizing repeated whitespace, removing hidden bidirectional marks, and trimming leading and trailing spaces. These operations reduce formatting noise without changing the linguistic form of the Arabic words. The articles and summaries are then tokenized using the AraT5 tokenizer. Tokenization converts text into token IDs that the model can process.
- The maximum input length is 1024 tokens and the maximum target length is 128 tokens. Padding is not applied inside the preprocessing function; instead, dynamic padding is applied later by DataCollatorForSeq2Seq during batching. This choice avoids unnecessary padding and reduces memory use during training.

4.5 Model Architecture

- The proposed model is based on the Transformer encoder-decoder architecture. The encoder reads the Arabic input text and produces contextual representations, while the decoder uses these representations to generate the summary token by token. Self-attention captures relationships inside the input sequence, and cross-attention connects the generated tokens with relevant information from the source text.
- The practical model is UBC-NLP/AraT5v2-base-1024 [6]. This model was selected because it is pretrained for Arabic sequence-to-sequence tasks and can be fine-tuned for abstractive summarization. The model selection is also consistent with the project goal of building a practical assistant rather than comparing many architectures.

4.6 Training Configuration

- The correct training configuration from the notebook is as follows. The model was fine-tuned for 15 epochs using a learning rate of $3e-5$. The train batch size was 2, the evaluation batch size was 2, gradient accumulation steps were set to 4, weight decay was 0.01, and early stopping patience was 2. Evaluation and saving were performed at the end of each epoch. The best model was selected based on validation performance.
- The previous values that stated 3 epochs and a learning rate of $5e-5$ must not be used in the final thesis because they do not match the final notebook implementation. Gradient accumulation was used to simulate a larger effective batch without exceeding GPU memory limits. Early stopping was used to reduce the risk of unnecessary training after validation performance stopped improving.

Table 4.2: Actual training configuration used in the notebook implementation.

Parameter	Value
Model	UBC-NLP/AraT5v2-base-1024
Dataset	karimraouf/Arabic-Summarization-Dataset-AsDs
Epochs	15
Learning Rate	$3e-5$
Train Batch Size	2
Evaluation Batch Size	2
Gradient Accumulation Steps	4
Weight Decay	0.01
Early Stopping Patience	2
Max Input Length	1024
Max Target Length	128

4.7 Summary Generation

After fine-tuning, summaries are generated using beam search. Beam search keeps several candidate summaries at each decoding step and selects the sequence with the best overall score. This is more stable than greedy decoding because it does not commit to only the single most likely next token at each step. The correct generation settings are `num_beams = 4`, `min_summary_length = 15`, `max_summary_length = 128`, `length_penalty = 1.0`, and `no_repeat_ngram_size = 3`.

Table 4.3: Generation parameters used for summary decoding.

Generation Parameter	Value
num_beams	4
min_length	15
max_length	128
length_penalty	1.0
no_repeat_ngram_size	3

4.8 Project Pipeline

The system pipeline begins with either Arabic text input or a text-based PDF file. If a PDF is provided, the system extracts text from it. The text is then cleaned, tokenized, divided into chunks if it exceeds the model length limit, summarized chunk by chunk, merged, and summarized again if needed. Finally, the output summary is displayed through the Gradio interface. This pipeline ensures that the system can handle both direct text and long document input while respecting the model input length.

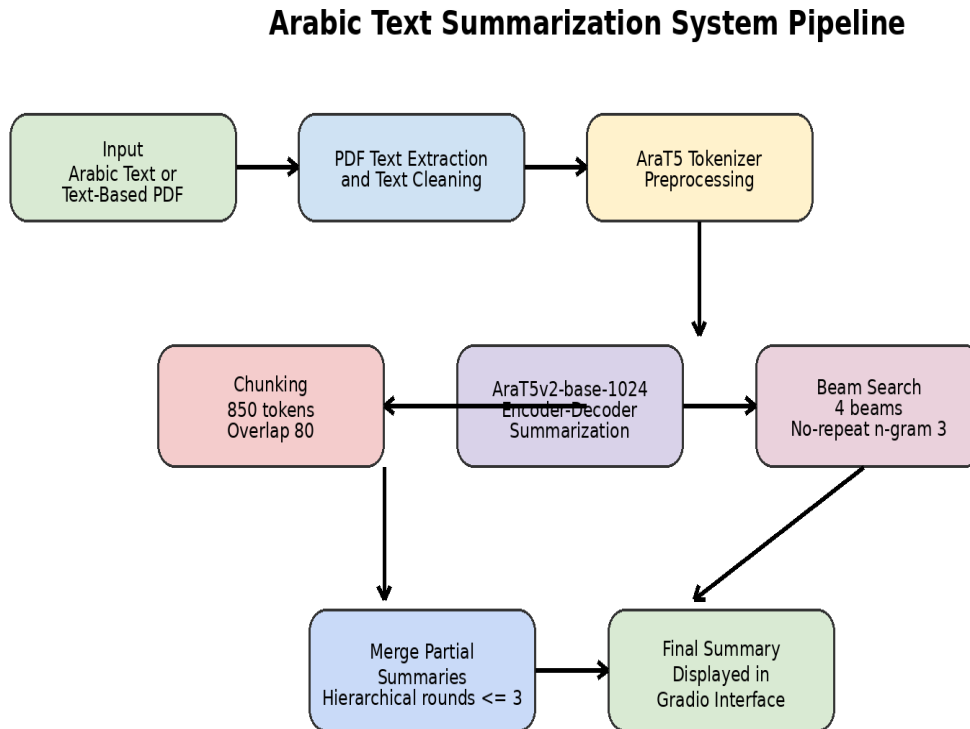


Figure 4.1: End-to-end pipeline of the Arabic text summarization system.

4.9 Long-Text Summarization

The long-text component is a necessary part of the system because AraT5v2 has a maximum input length. The implemented algorithm splits the input into overlapping token chunks, summarizes each chunk, merges the partial summaries, and repeats the process when the merged result is still too long. The overlap of 80 tokens reduces the chance of losing context at chunk boundaries.

Table 4.4: Long-text chunking configuration.

Parameter	Value
Chunk token size	850
Chunk overlap	80
Maximum hierarchical rounds	3
Supported PDF type	Text-based PDF only
Unsupported PDF type	Scanned PDF without OCR

4.10 Practical Interface

- A Gradio interface was built to make the model usable by non-specialist users. The interface provides two tabs: one for Arabic text summarization and one for text-based PDF summarization. This practical component is important because many studies stop at evaluation results, while this project also demonstrates how the trained model can be used interactively.
- It is important to state that scanned PDFs are not supported because OCR was not implemented in the project code. Therefore, the supported PDF files must contain extractable text. Adding OCR remains a future improvement and should not be claimed as a current feature.

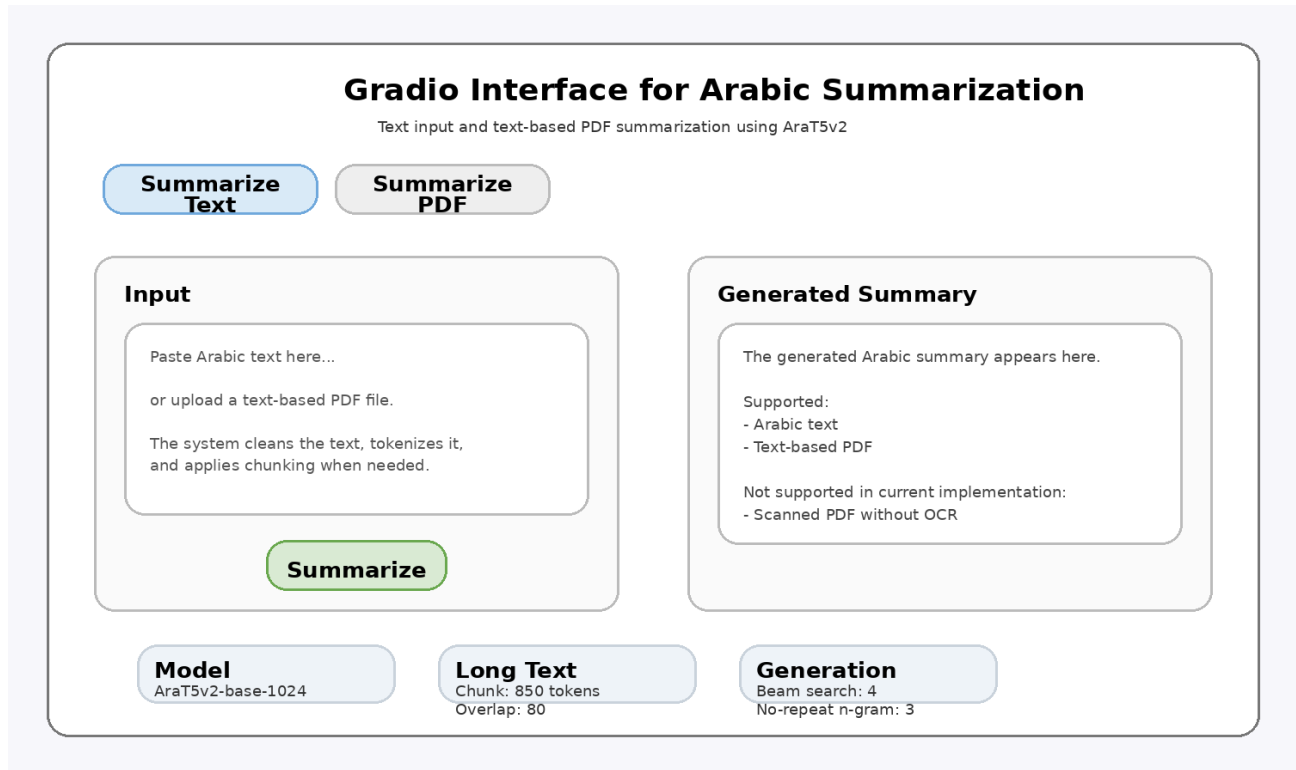


Figure 4.2: Representative Gradio interface for Arabic text and text-based PDF summarization.

4.11 Implementation Challenges and Solutions

- Long input texts were handled using chunking with 850 tokens per chunk and 80 tokens of overlap.
- Repeated phrases were reduced using `no_repeat_ngram_size = 3` during generation.
- Limited training data was addressed by using transfer learning from AraT5v2.
- Computational cost was reduced by using Google Colab GPU when available and small batch sizes with gradient accumulation.
- Text-based PDF summarization was supported, but scanned PDFs remain a future improvement because OCR is not implemented.

4.12 Detailed Explanation of the Notebook Code

- The implementation notebook is organized as an end-to-end pipeline rather than isolated code cells. Each section performs one clear task: preparing the environment, loading data, configuring the model, preprocessing the Arabic text, training the model, saving the outputs, evaluating the generated summaries, and finally deploying the assistant through Gradio. This organization makes the project easier to reproduce and easier to explain during the defense.
- The first part installs and imports the required libraries. The important libraries are transformers for loading AraT5v2 and using Seq2SeqTrainer, datasets for loading the ASDs dataset, evaluate for automatic metrics, sentence-transformers for semantic similarity, pypdf for extracting text from text-based PDF files, and gradio for building the interactive interface. PyTorch is the backend used for model training and generation.
- The second part defines the configuration class CFG. This class is important because it keeps all key values in one place: model name, dataset name, maximum input length, maximum target length, number of epochs, learning rate, batch sizes, gradient accumulation, generation parameters, chunk size, overlap, and output directories. This design reduces confusion because the thesis, the code, and the experimental discussion all refer to the same configuration values.
- The third part loads the Arabic Summarization Dataset (ASDs) from Hugging Face using its official train, validation, and test splits. The code does not create a new random split; therefore, the thesis correctly reports 2075 training samples, 259 validation samples, and 260 test samples. Each sample includes the article and the reference summary, which are the two fields used during model fine-tuning.
- The fourth part performs the implemented preprocessing. The function `clean_arabic_text` converts values to strings, removes repeated whitespace, removes hidden right-to-left and left-to-right marks, and trims leading and trailing spaces. The code intentionally does not claim advanced Arabic normalization such as full diacritic removal or OCR-based cleaning because these operations were not implemented. After cleaning, the tokenizer converts articles and summaries into token IDs suitable for AraT5v2.

- The fifth part prepares training through `DataCollatorForSeq2Seq` and `Seq2SeqTrainingArguments`. Dynamic padding is applied by the collator during batching, which is more memory-efficient than padding all examples to the maximum length during preprocessing. The training arguments specify evaluation at each epoch, saving checkpoints at each epoch, using beam search during prediction, loading the best model at the end, and selecting the best checkpoint according to validation ROUGE-L.
- The sixth part trains the model using `Seq2SeqTrainer`. During training, the model receives article tokens as input and learns to generate summary tokens as output. The loss is calculated between the predicted token distribution and the reference summary tokens. This is why the loss is not an accuracy percentage; it is a training objective that guides weight updates. After training, the notebook saves the model, tokenizer, training metrics, plots, and predictions under the project directory.
- The seventh part defines inference functions. The `summarize_text` function cleans the input, tokenizes it, sends it to the trained model, and generates the output summary using beam search. The generation settings control the output length and reduce repetition through `no_repeat_ngram_size = 3`. The result is decoded back into Arabic text and cleaned again before being displayed to the user.
- The eighth part handles long inputs using chunking. If the text is longer than the model can safely process, the code divides it into overlapping token chunks of 850 tokens with an overlap of 80 tokens. Each chunk is summarized separately. The partial summaries are then merged and summarized again when needed. This hierarchical strategy allows the assistant to process long Arabic text without exceeding the 1024-token input limit of `AraT5v2`.
- The ninth part handles PDF input. The code uses `pypdf` to extract text from PDF pages, then sends the extracted text to the same long-text summarization pipeline. This means the system supports text-based PDFs only. If the PDF is scanned as images, `pypdf` cannot extract readable text, and the interface returns a message explaining that OCR is required. Therefore, scanned PDF support is correctly listed as future work, not as a current feature.

- The final part evaluates and deploys the system. The evaluation code generates summaries for the official test split and computes seven metrics: ROUGE-1, ROUGE-2, ROUGE-L, BERTScore, METEOR, chrF, and Semantic Similarity. The Gradio interface then wraps the inference functions into two tabs: one for direct Arabic text and one for text-based PDF files. This makes the project not only a trained model, but also a practical assistant that can be demonstrated live.

Table 4.5: Detailed code workflow and its role in the implemented system.

Code Stage	What the code does	Why it is important
Environment setup	Installs and imports Transformers, Datasets, Evaluate, PyTorch, pypdf, Gradio, and SentenceTransformers.	Provides the libraries needed for training, evaluation, PDF extraction, and the interface.
Configuration	Defines CFG values such as model name, dataset name, lengths, hyperparameters, generation settings, and chunking values.	Keeps the experiment reproducible and keeps thesis values consistent with the code.
Dataset loading	Loads ASDs with official train, validation, and test splits.	Avoids unsupported random splitting and makes results easier to reproduce.
Cleaning and tokenization	Normalizes whitespace, removes hidden direction marks, tokenizes article and summary fields.	Prepares Arabic text for AraT5v2 while avoiding preprocessing claims not implemented in code.
Training	Uses Seq2SeqTrainer, AdamW behavior, validation each epoch, checkpoints, early stopping, and best model selection.	Fine-tunes the pretrained model for Arabic summarization under Colab memory limits.
Generation	Uses beam search, min/max length, length penalty, and no-repeat n-gram control.	Produces more stable summaries and reduces repeated phrases.
Long-text chunking	Splits long text into overlapping chunks, summarizes each chunk,	Allows long Arabic documents to be summarized despite the 1024-token limit.

Summarizing texts in Arabic

	and applies hierarchical resummation.	
PDF summarization	Extracts text using pypdf and passes it to the long-text pipeline.	Supports text-based PDF files while clearly excluding scanned PDFs without OCR.
Evaluation	Calculates ROUGE, BERTScore, METEOR, chrF, and Semantic Similarity on the test split.	Combines lexical and semantic metrics to analyze abstractive summarization quality.
Gradio interface	Creates two tabs for text summarization and PDF summarization.	Turns the trained model into a usable assistant for demonstration and practical use.

Chapter 5: Results and Evaluation — الفصل الخامس: نتائج البحث

5.1 Tools and Evaluation Methods

The experiments were implemented using Python, PyTorch, Hugging Face Transformers, Hugging Face Datasets, Evaluate, SentenceTransformers, pypdf, and Gradio in Google Colab. GPU acceleration was used when available. The evaluation focuses on metrics that were actually calculated in the notebook: ROUGE-1, ROUGE-2, ROUGE-L, BERTScore, METEOR, chrF, and Semantic Similarity. Precision, Recall, and F1 Score as classification metrics were removed from the thesis because they were not computed directly in the final notebook for the generative summarization task.

The following equations were added because the official thesis template requires the evaluation criteria and their mathematical meaning. The equations are presented in a simplified form suitable for a graduation project discussion. They explain what the metrics measure without claiming that every internal implementation detail of the libraries is manually coded in the notebook.

Equation (5.1): ROUGE-N recall-based overlap.

$$\text{ROUGE-N} = (\text{Number of matching n-grams between generated summary and reference summary}) / (\text{Number of n-grams in the reference summary})$$

Equation (5.2): ROUGE-L longest common subsequence recall.

$$\text{ROUGE-L} = \text{LCS}(\text{Generated Summary}, \text{Reference Summary}) / \text{Length}(\text{Reference Summary})$$

Equation (5.3): BERTScore F1 formulation.

$$\text{BERTScore-F1} = 2 \times (\text{Precision_BERT} \times \text{Recall_BERT}) / (\text{Precision_BERT} + \text{Recall_BERT})$$

Equation (5.4): Semantic similarity using cosine similarity.

$$\text{Cosine Similarity} = (A \cdot B) / (\|A\| \times \|B\|)$$

Table 5.1: Evaluation metrics and equations used in the project.

Metric	Main Idea	Equation / Interpretation
ROUGE-1	Unigram overlap	Uses Equation (5.1) with $n = 1$ to measure word-level content overlap.
ROUGE-2	Bigram overlap	Uses Equation (5.1) with $n = 2$ to measure phrase-level overlap.

ROUGE-L	Longest common subsequence	Uses Equation (5.2) to capture structural overlap and word order.
BERTScore	Embedding-based semantic similarity	Uses contextual embeddings and reports F1 as in Equation (5.3).
METEOR	Flexible overlap metric	Uses flexible unigram matching with precision/recall weighting and penalty; useful beyond exact overlap.
chrF	Character n-gram F-score	Uses character-level precision and recall, useful for morphologically rich languages.
Semantic Similarity	Meaning-level closeness	Uses sentence embeddings and cosine similarity as in Equation (5.4).

5.2 Results of the Proposed Model

The final evaluation was conducted on the full official test split containing 260 samples. The final metrics obtained from the notebook are presented in Table 5.2. These values should be reported exactly as obtained from the final notebook output and should not be replaced by values from earlier training attempts or quick tests.

Table 5.2: Final test results obtained on the official ASDs test split.

Metric	Value
ROUGE-1	7.4982
ROUGE-2	0.3846
ROUGE-L	7.5458
BERTScore	77.7619
METEOR	26.8728
chrF	39.6919
Semantic Similarity	83.5240

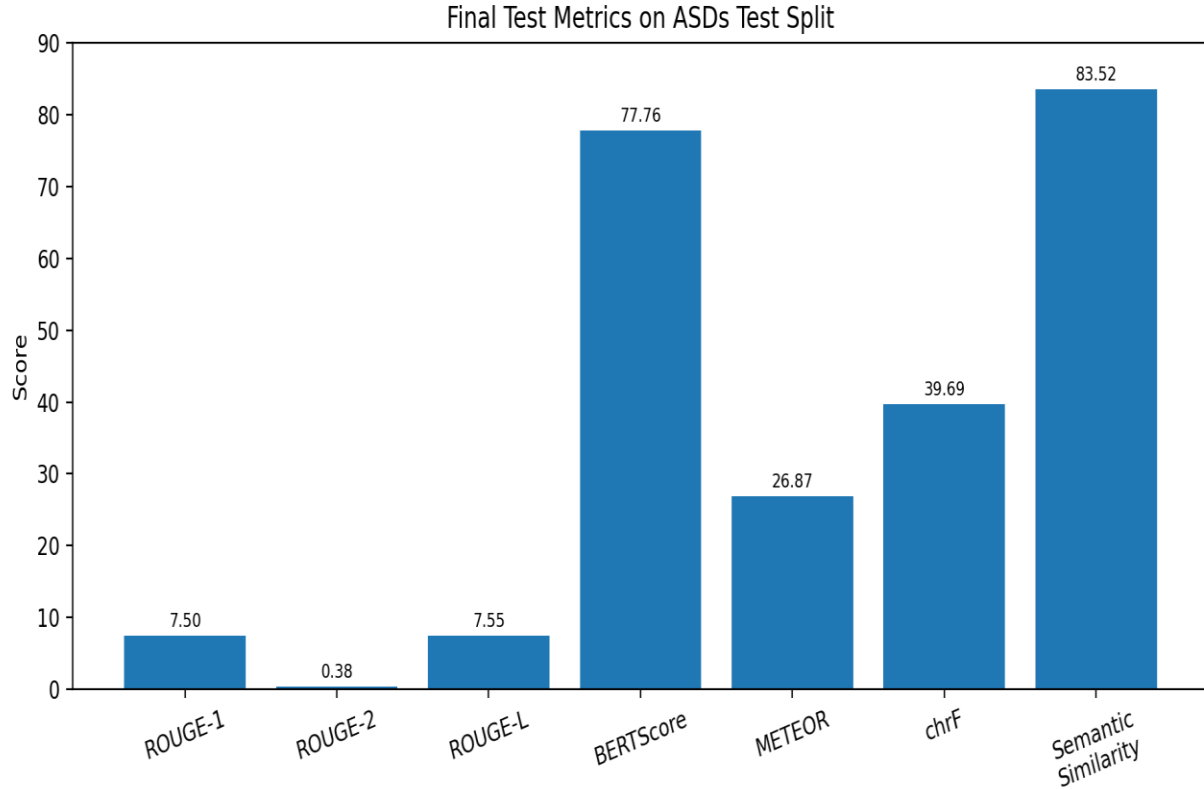


Figure 5.1: Bar chart of the final evaluation metrics.

These results show that the generated summaries preserve a meaningful semantic relationship with the references, especially according to BERTScore and Semantic Similarity. The lower ROUGE values indicate that the generated summaries do not always share the same surface wording as the references, which is common in abstractive summarization. This does not mean that the model failed completely; rather, it suggests that the generated wording differs from the reference wording and that further fine-tuning and dataset expansion are needed.

5.3 Result Analysis

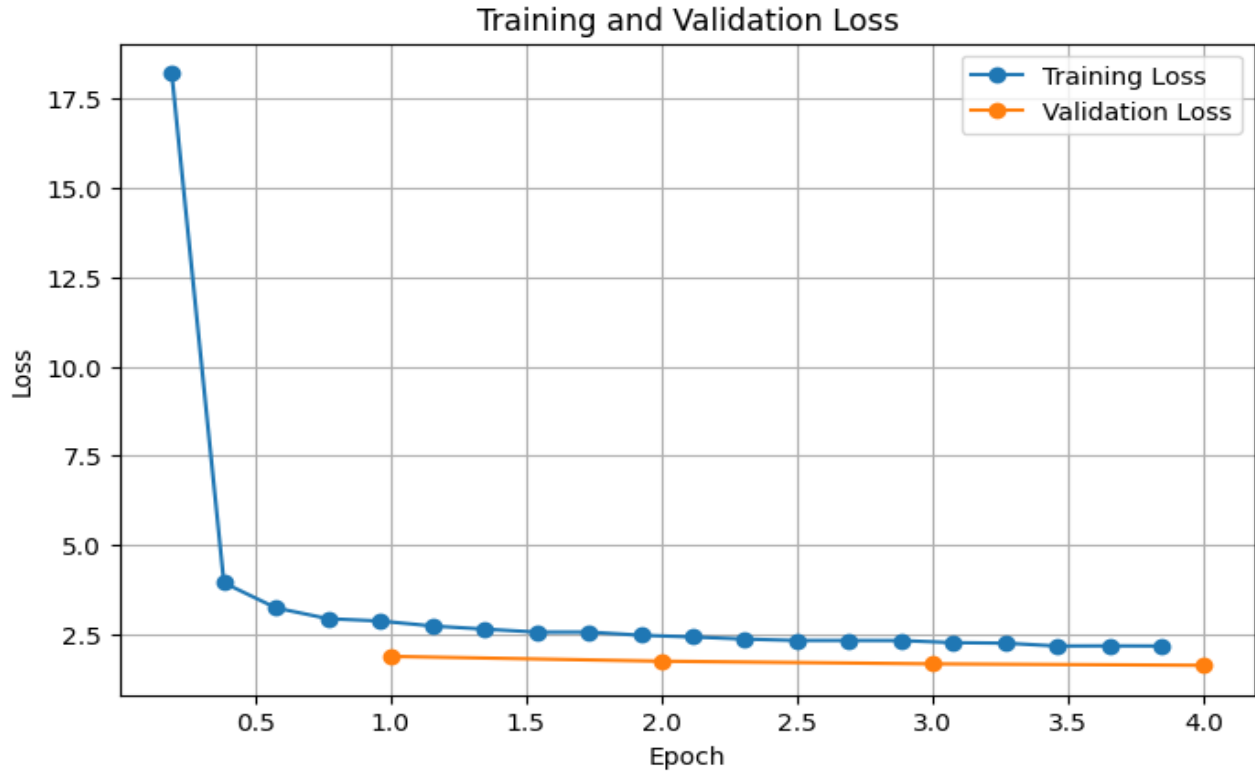


Figure 5.2: Training and validation loss curve during fine-tuning.

Qualitative testing showed that the model can produce understandable Arabic summaries and reduce redundancy. The use of beam search improved generation stability, while `no_repeat_ngram_size = 3` helped reduce repeated phrases. The chunking mechanism also allowed the system to summarize input longer than the model's maximum context size.

However, the model may still face difficulties with rare technical terms, uncommon proper names, and texts containing many dense ideas in a small number of sentences. The chunking approach may also lose some cross-chunk context if the document structure is highly interconnected. This limitation is expected because each chunk is summarized separately before the hierarchical merging step.

5.4 Practical Demonstration

The notebook includes a long-text test in which the input exceeded the model's maximum sequence length and was therefore split into two chunks. The resulting summary demonstrates that the hierarchical chunking mechanism can process long Arabic input without directly exceeding the model input limit.

Table 5.3: Qualitative demonstration of long-text summarization.

<i>Input Type</i>	<i>System Behavior</i>	<i>Generated Output Summary</i>
<i>Long Arabic text</i>	The input was split into 2 chunks, summarized separately, then merged.	The generated summary explained that technology has become an essential part of modern life, affecting education, healthcare, communication, and work. It also mentioned benefits such as access to information, distance learning, smart diagnosis, and electronic health records, while noting privacy, data security, and reduced direct communication as challenges.

5.5 Comparison of Proposed Experiments

The current implementation focuses on one practical model: AraT5v2-base-1024. Therefore, the thesis should not claim that several models were experimentally compared unless those experiments are actually added. A future comparison can include AraBART, mT5, or a non-fine-tuned baseline using the same dataset, preprocessing, and evaluation script. For the current version, comparison is limited to analytical discussion of the implemented configuration and the effect of chunking and controlled generation settings.

5.6 Comparison with Previous Work

A direct numerical comparison with previous studies is difficult because datasets, preprocessing, and evaluation settings differ. Therefore, the comparison is analytical unless the same dataset and evaluation protocol are reproduced. The project aligns with previous research that emphasizes pretrained Arabic sequence-to-sequence models and long-text handling strategies, while adding a practical Gradio-based assistant for text and text-based PDF summarization.

Table 5.4: Analytical comparison between the proposed system and selected previous work.

Work	Model / Method	Strength	Difference from this thesis
Kamal Eddine et al. (2022) [3]	AraBART	Strong Arabic abstractive baseline with end-to-end pretrained seq2seq architecture.	Focuses on a pretrained model and benchmarks rather than a practical PDF/text assistant.
Abdul Salam et al. (2024) [2]	MSG-ATS semantic graph	Captures syntactic, semantic, and contextual information through graph-based representation.	More complex graph-based framework; the current thesis uses AraT5v2 fine-tuning.

Summarizing texts in Arabic

Merniz et al. (2021) [5]	Thematic annotation + modified PageRank	Good thematic coverage and redundancy reduction for multi-document summarization.	Extractive and multi-document oriented, while this thesis is abstractive and user-interface oriented.
Masri et al. (2024) [4]	AraBART / MT5 / AraT5 / mBART50	Educational Arabic textbook summarization; closest to the educational motivation of this thesis.	Uses a small textbook dataset and compares models; this thesis builds a Gradio assistant with long-text chunking.
This thesis	AraT5v2 + Chunking + Gradio	Practical Arabic assistant with long-text and text-based PDF support.	Needs stronger ROUGE scores, more data, OCR support, and structured human evaluation.

Chapter 6: Conclusion — الفصل السادس: الخاتمة

6.1 Conclusions

- This project demonstrates that AraT5v2-base-1024 can be used to build a practical Arabic abstractive summarization assistant. The system combines fine-tuning, long-text chunking, controlled beam-search generation, multi-metric evaluation, and a Gradio interface. The implemented system is aligned with the project objective because it accepts Arabic text and text-based PDF input and generates concise Arabic summaries through a usable interface.
- The results indicate that the model can generate semantically coherent summaries, although surface-level overlap metrics such as ROUGE still require improvement. The project also shows that chunking is necessary for processing long Arabic texts and text-based PDF files that exceed the model input limit. The combination of lexical and semantic metrics provides a more balanced evaluation than reporting ROUGE alone.
- The final implementation corrected the earlier methodological inconsistencies: training was performed for 15 epochs, the learning rate was $3e-5$, the official dataset splits were used, Precision/Recall/F1 were removed from the final results, and PDF support was limited to text-based PDF files only. These corrections make the thesis consistent with the code and reduce the risk of unsupported claims during the defense.

6.2 Challenges and Future Work

- Expand the dataset with more educational, scientific, and technical Arabic content.
- Add OCR support to process scanned PDF files.
- Experiment with longer-context models such as Longformer-style architectures or newer Arabic LLMs.
- Compare AraT5v2 with AraBART, mT5, and non-fine-tuned baselines using the same dataset.
- Add structured human evaluation for fluency, faithfulness, coverage, and coherence.
- Improve faithfulness and reduce hallucination using verification-based techniques or retrieval-based grounding.
- Deploy the system as a web application or Hugging Face Space for practical use.

6.3 Final Summary

The thesis confirms that combining Arabic pretrained Transformer models with appropriate preprocessing, chunking, and semantic evaluation can support the development of useful Arabic summarization tools. The proposed system represents a practical foundation for educational and research-oriented Arabic knowledge management. Its current version is not the final possible solution for Arabic summarization, but it is a reproducible and extensible baseline that can be improved by adding more data, stronger models, OCR support, and human evaluation.

Appendix A: Notebook Implementation Summary

The implementation notebook was written as a complete workflow for training, testing, and deploying an Arabic summarization assistant. It does not only call a ready summarization model; it loads the dataset, prepares the text, fine-tunes AraT5v2, saves the trained model, reloads it for inference, evaluates it using multiple metrics, supports long-text chunking, extracts text from text-based PDFs, and exposes the functions through a Gradio interface.

The central component of the notebook is the CFG class. This class makes the implementation easy to audit because all important values are visible in one place: UBC-NLP/AraT5v2-base-1024 as the model, karimraouf/Arabic-Summarization-Dataset-AsDs as the dataset, 1024 tokens as the maximum input length, 128 tokens as the maximum target length, 15 epochs, learning rate 3e-5, batch size 2, gradient accumulation 4, beam size 4, chunk size 850, overlap 80, and 3 maximum hierarchical summarization rounds.

- Model: UBC-NLP/AraT5v2-base-1024.
- Dataset: karimraouf/Arabic-Summarization-Dataset-AsDs.
- Input and target lengths: 1024 tokens and 128 tokens.
- Training: 15 epochs, learning rate 3e-5, batch size 2, gradient accumulation 4.
- Generation: beam search with 4 beams and no-repeat n-gram size 3.
- Long text handling: chunk size 850 tokens, overlap 80 tokens, and up to 3 hierarchical rounds.

The notebook separates training and inference. During training, Seq2SeqTrainer updates the model weights using article-summary pairs. During inference, the saved model is reloaded from the model directory and used by `summarize_text`, `summarize_long_text`, and `summarize_pdf`. This separation is important because the project can be demonstrated later without repeating the full training process.

A.1 Environment and Libraries

The first notebook cell installs the main libraries. Transformers provides the model, tokenizer, Seq2SeqTrainer, Seq2SeqTrainingArguments, and generation utilities. Datasets loads the ASDs dataset. Evaluate loads ROUGE, METEOR, chrF, and BERTScore. SentenceTransformers is used for semantic similarity. pypdf extracts text from text-based PDF files. Gradio provides the final interface.

A.2 Project Directories

The notebook creates a project directory and subdirectories for the saved model, results, plots, and predictions. When Google Drive is enabled, these folders are created under the

user Drive path. This is useful because checkpoints, metrics, and saved models remain available after the Colab session ends.

A.3 Dataset Use

The dataset is loaded from Hugging Face and the code uses its official splits. The thesis must therefore report the exact official numbers rather than an 80/20 manual split. The article column is used as source input and the summary column is used as the target output.

A.4 Preprocessing Function

The preprocessing function is deliberately simple and transparent. It converts values to strings, removes repeated whitespace, removes hidden bidirectional marks, and trims the text. Then the tokenizer converts the text into token IDs. The code uses truncation to respect the maximum lengths and leaves padding to DataCollatorForSeq2Seq.

A.5 Training Logic

The training cell builds DataCollatorForSeq2Seq and Seq2SeqTrainer. The model is trained for 15 epochs with learning rate $3e-5$. Because the batch size is small, gradient accumulation is used to improve the effective batch size without exceeding memory. Evaluation and saving happen at each epoch, and the best model is selected using validation ROUGE-L.

A.6 Inference Logic

The `summarize_text` function is decorated with `torch.no_grad`, which disables gradient computation during inference. This saves memory and speeds up generation. The function tokenizes the cleaned input, calls `model.generate` with beam search, decodes the generated token IDs, and returns the final Arabic summary.

A.7 Long-Text Logic

The `split_text_by_tokens` function works at token level, not only at character level. This is important because the model limit is measured in tokens. The function creates overlapping chunks so that information near boundaries is less likely to be lost. The `summarize_long_text` function summarizes chunks, merges partial summaries, and repeats summarization if the merged text is still too long.

A.8 PDF Logic

The PDF function uses PdfReader and `page.extract_text`. Therefore, the implementation supports PDFs that already contain selectable/extractable text. Scanned PDFs are images

and cannot be handled by this code without OCR. The interface correctly informs the user when no text can be extracted.

A.9 Evaluation Logic

The final evaluation generates predictions on the official test split and compares them with reference summaries. ROUGE measures lexical overlap, BERTScore and Semantic Similarity measure meaning-level similarity, METEOR provides flexible matching, and chrF evaluates character-level similarity. This combination is more suitable for abstractive Arabic summarization than relying on ROUGE alone.

A.10 Interface Logic

The Gradio interface connects the user inputs to the inference functions. The first tab summarizes direct Arabic text. The second tab accepts a PDF file and calls `summarize_pdf`. This makes the project practical because the trained model can be used through a simple interface rather than only through code cells.

Appendix B: Reproducibility and Deployment Notes

To reproduce the project, the user should run the notebook in Google Colab with GPU enabled. The required Python libraries include transformers, datasets, evaluate, accelerate, sentencepiece, rouge_score, bert-score, nltk, pypdf, gradio, sentence-transformers, and PyTorch. The notebook saves the model, tokenizer, metrics, plots, and predictions under the project directory in Google Drive.

The interface was implemented with Gradio to support two practical use cases: summarizing direct Arabic text and summarizing text-based PDF files. The PDF component extracts text using pypdf and then passes the extracted text to the same summarization pipeline. Scanned PDFs are excluded because they require OCR, which was not implemented in the current version.

- Use a GPU runtime when fine-tuning the model to reduce training time.
- Keep the saved model directory unchanged when testing inference after training.
- Use the official dataset splits to keep the results consistent with the thesis.
- Do not report metrics that are not computed in the notebook.
- When demonstrating PDF summarization, use text-based PDFs rather than scanned images.

For future deployment, the Gradio interface can be hosted using a local server, Google Colab public sharing, or a Hugging Face Space. A permanent deployment should include input length warnings, file type validation, and a clear note that OCR is not yet supported.

Appendix C: Consistency Notes Between Thesis and Implementation

This appendix documents the consistency between the final thesis and the practical notebook implementation. It is included to make the methodological choices clear and reproducible.

- The implemented model is AraT5v2-base-1024, while AraBART is discussed only as related work.
- The final training configuration uses 15 epochs and a learning rate of $3e-5$.
- The official ASDs dataset splits are used: Train = 2075, Validation = 259, and Test = 260.
- The reported evaluation metrics are ROUGE-1, ROUGE-2, ROUGE-L, BERTScore, METEOR, chrF, and Semantic Similarity.
- The interface supports text-based PDF files only; scanned PDFs require OCR and remain future work.
- The generation configuration uses beam search with 4 beams, length penalty 1.0, and no-repeat n-gram size 3.

These consistency notes ensure that the thesis content accurately reflects the implementation and avoids unsupported experimental claims.

References

Only the nine references below are used in the final thesis. Extra general references that were previously added for metrics, optimizers, or standard Transformer background were removed to keep the reference list aligned with the sources provided for the project and the two code-related sources actually used by the notebook.

- [1] M. Magdy, S. O. Slim, and E. H. Mohamed, “Exploring Advances in Arabic Long-Text Summarization Strategies: A Survey,” *Informatics Bulletin, Faculty of Computers and Artificial Intelligence, Helwan University*, vol. 6, no. 2, July 2024.
- [2] M. Abdul Salam, M. Aldawsari, M. Gamal, H. F. A. Hamed, and S. Sweidan, “MSG-ATS: Multi-Level Semantic Graph for Arabic Text Summarization,” *IEEE Access*, vol. 12, pp. 118773-118784, 2024, doi: 10.1109/ACCESS.2024.3441489.
- [3] M. Kamal Eddine, N. Tomeh, N. Habash, J. Le Roux, and M. Vazirgiannis, “AraBART: a Pretrained Arabic Sequence-to-Sequence Model for Abstractive Summarization,” *Proceedings of the Seventh Arabic Natural Language Processing Workshop (WANLP)*, pp. 31-42, 2022.
- [4] S. Masri, Y. Raddad, F. Khandaqji, H. I. Ashqar, and M. Elhenawy, “Transformer Models in Education: Summarizing Science Textbooks with AraBART, MT5, AraT5, and mBART,” *arXiv:2406.07692*, 2024.
- [5] A. Merniz, A. Habacha Chaibi, and H. Hajjami Ben Ghezala, “Multi-document Arabic Text Summarization based on Thematic Annotation,” *Proceedings of the 16th International Conference on Software Technologies (ICSOFT)*, pp. 639-644, 2021, doi: 10.5220/0010557906390644.
- [6] Y. Zhang, H. Jin, D. Meng, J. Wang, and J. Tan, “A Comprehensive Survey on Automatic Text Summarization with Exploration of LLM-Based Methods,” *arXiv:2403.02901v3*, 2025.
- [7] Elicit, “What are the most effective extractive and abstractive text summarization techniques for Arabic language documents published between 2018 and 2026?” *Elicit Research Report*, 2026.
- [8] UBC-NLP, “AraT5v2-base-1024 Model Card,” *Hugging Face*, used as the pretrained model source in the implementation, accessed 20 April 2026.
- [9] K. Raouf, “Arabic-Summarization-Dataset-AsDs,” *Hugging Face Datasets*, used as the dataset source in the implementation, accessed 20 April 2026.